객체지향언어1(7반)

과제 2 - 5장 함수와 참조

- 과제명: 실습문제 5장 6번, 8번 풀기
- 학번: 2371172
- 이름: 안태호
- 트랙명: 빅데이터 트랙, 웹 공학 트랙
- 담당 교수명: 황기태

1. 실습문제 6번 소스코드

Polygon.h

```
#ifndef POLYGON_H //POLYGON_H가 정의되어 있지 않다면
#define POLYGON_H //POLYGON_H를 정의함(중복 포함 방지)

class Polygon { //Polygon 클래스 선언
	int size = 0; //다각형 점의 개수를 저장하는 멤버 변수
	int* xList, * yList; //x좌표와 y좌표 배열을 가리킬 포인터 변수 선언
public: //아래 멤버들을 public으로 접근 지정
	Polygon(int size); //매개변수가 있는 생성자 선언, 동적 배열 할당에 사용
	~Polygon();//소멸자 선언, 객체 소멸 시 동적 메모리 해제
	void read();//사용자로부터 좌표 값을 입력받는 함수 선언
	bool get(int n, int& x, int& y);//n번째 점의 좌표를 참조 매개변수로 반환하는 함수 선언
};
#endif // 조건부 컴파일 종료
```

Polygon.cpp

```
#include "Polygon.h" //Polygon 클래스 선언 헤더 포함
#include <iostream> // 입출력 기능을 위한 표준 헤더 파일
using namespace std; // std 이름공간에 선언된 모든 이름에 std를 생략함
//매개변수가 있는 생성자 정의
Polygon::Polygon(int size) {
	this->size = size; // 매개변수 size 값을 멤버 변수 size에 저장
	xList = new int[size];// size 크기만큼 x좌표를 저장할 배열을 동적 할당
```

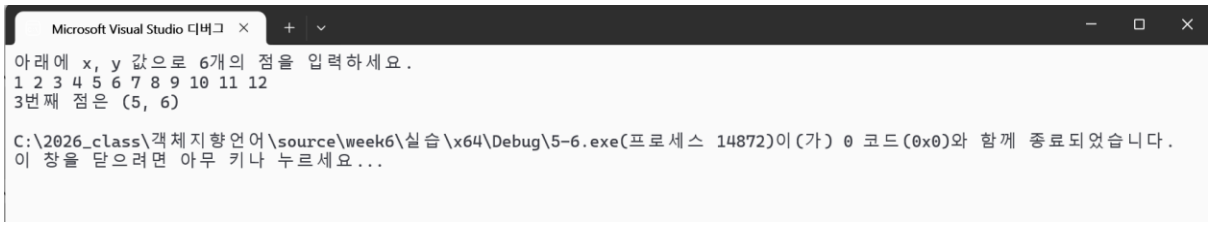
```
	yList = new int[size];// size 크기만큼 y좌표를 저장할 배열을 동적 할당
}
//소멸자 정의
Polygon::~Polygon() {
	delete[] xList; //동적 할당된 x좌표 배열 메모리 해제
	delete[] yList; //동적 할당된 y좌표 배열 메모리 해제
}
//좌표를 입력받는 함수 정의
void Polygon::read() {
	cout << "아래에 x, y 값으로 " << size << "개의 점을 입력하세요." << endl;
	for (int i = 0; i < size; i++) { // size만큼 반복하여 입력받음
		cin >> xList[i] >> yList[i]; // 입력된 값을 배열의 i번째 요소에 저장
	}
}
// 특정 번호의 점 좌표를 반환하는 함수 정의
bool Polygon::get(int n, int& x, int& y) {
	//입력된 번호가 유효한 범위(1~size)를 벗어나면 false 반환
	if (n < 1 || n > size)
		return false;
	x = xList[n - 1]; //배열의 n-1 인덱스에 저장된 x값을 참조 변수 x에 저장
	y = yList[n - 1]; //배열의 n-1 인덱스에 저장된 y값을 참조 변수 y에 저장
	return true; //성공적으로 값을 전달했으므로 true 반환
}
```

main.cpp

```
#include "Polygon.h" // Polygon 클래스 선언이 담긴 헤더 포함
#include <iostream> // 입출력 기능을 위한 표준 헤더 파일
using namespace std; // std 이름공간에 선언된 모든 이름에 std를 생략함


int main() {
	// 매개변수 생성자를 호출하여 점이 6개인 poly 객체 생성
	Polygon poly(6);
	poly.read(); //poly 객체의 read 함수를 호출하여 점의 좌표를 입력받음
	int n = 3, x, y;// 점의 번호와 좌표를 저장할 변수 선언
	//get 함수를 호출하여 n번째 점의 좌표를 가져오고 성공 여부를 res에 저장
	bool res = poly.get(n, x, y);
	if (res) // 점을 성공적으로 찾았다면 실행
		cout << n << "번째 점은 " << "(" << x << ", " << y << ")" << endl;
	else // 범위를 벗어나 점을 찾지 못했다면 실행
		cout << n << "번째 점은 없습니다." << endl;
	return 0; //프로그램을 종료하고 운영체제에 0을 반환
}
```

실행결과



2. 실습문제 8번 소스코드

Fitness.h

```
#ifndef FITNESS_H //FITNESS_H가 정의되어 있지 않다면
#define FITNESS_H //FITNESS_H를 정의함(중복 포함 방지)
#include <string> // 문자열 처리를 위한 표준 헤더 파일

class Fitness { // Fitness 클래스 선언
	int men = 0;//남자 회원 수를 저장하고 0으로 초기화하는 멤버 변수
	int women = 0;// 여자 회원 수를 저장하고 0으로 초기화하는 멤버 변수
public: // 아래 멤버들을 public으로 접근 지정
	// 자신을 참조로 반환하여 연쇄 호출이 가능한 arrive 함수 선언
	Fitness& arrive(std::string gender, int arriveCount);
	// 특정 성별의 회원 수를 참조 매개변수로 전달하는 함수 선언
	void count(std::string gender, int& memberCount);
	// 특정 성별 멤버 변수의 참조를 직접 반환하여 수정 가능한 함수 선언
	int& at(std::string gender);
};
#endif //조건부 컴파일 종료
```

Fitness.cpp

```
#include "Fitness.h" //Fitness 클래스 선언 헤더 포함
#include <iostream> //입출력 기능을 위한 표준 헤더 파일
#include <string> //문자열 처리를 위한 표준 헤더 파일
#include <cstdlib> // exit() 함수 사용을 위한 헤더 파일
using namespace std; //std 이름공간에 선언된 모든 이름에 std를 생략함

// 회원이 도착했을 때 성별에 맞게 인원수를 누적하는 함수 정의
Fitness& Fitness::arrive(string gender, int arriveCount) {
	if (gender == "남") //성별이 "남"인 경우
		men += arriveCount; // men 멤버 변수에 인원수 누적
	else if (gender == "여") // 성별이 "여"인 경우
		women += arriveCount; // women 멤버 변수에 인원수 누적
	else { // 잘못된 성별 입력시 에러 메시지 출력
		cout << "ERROR! arrive()에 잘못된 성별을 입력했습니다 (" << gender << ") "
			<< endl;
	}
	return *this; //현재 객체 자신의 참조를 반환
}
//특정 성별의 인원수를 참조 매개변수에 복사해주는 함수 정의
void Fitness::count(string gender, int& memberCount) {
	if (gender == "남") // 성별이 "남"인 경우
		memberCount = men; // memberCount에 men 값을 저장
	else if (gender == "여") // 성별이 "여"인 경우
		memberCount = women; //memberCount에 women 값을 저장
	else { // 잘못된 성별 입력 시 에러 메시지 출력 후 인원수를 0으로 초기화
		cout << "ERROR! count()에 잘못된 성별을 입력했습니다 (" << gender << ")"
```

```
                << endl;
            memberCount = 0;
        }
}
// 특정 성별 멤버 변수의 참조를 반환하는 함수 정의
int& Fitness::at(string gender) {
        if (gender == "남") // 성별이 "남"인 경우
            return men; // men 변수 자체의 참조를 반환
        else if (gender == "여") //성별이 "여"인 경우
            return women;//women 변수 자체의 참조를 반환
        else {//잘못된 성별 입력 시 참조 반환이 불가능하므로 에러 메시지 출력 후 종료
            cout << "ERROR! at()에 잘못된 성별을 입력했습니다 (" << gender << ")" <<
                endl;
            exit(1); //프로그램 비정상 종료
        }
}
```

main.cpp

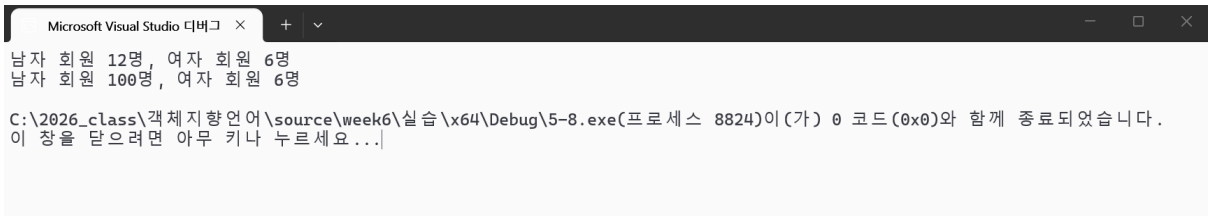
```
#include "Fitness.h"// Fitness 클래스 선언이 담긴 헤더 포함
#include <iostream>// 입출력 기능을 위한 표준 헤더 파일
using namespace std;// std 이름공간에 선언된 모든 이름에 std를 생략함

int main() {
        Fitness altong; // Fitness 객체 altong 생성
        //arrive 함수를 연속적으로 호출하여 남성 3명, 여성 6명, 남성 9명을 추가
        altong.arrive("남", 3).arrive("여", 6).arrive("남", 9);
```

```
    int m = 0, w = 0; // 회원 수를 담을 변수 m, w 선언 및 초기화
    altong.count("남", m);// count 함수를 호출하여 남자 회원 수를 m에 저장
    altong.count("여", w);// count 함수를 호출하여 여자 회원 수를 w에 저장
    //현재 저장된 남자와 여자 회원 수를 출력
    cout << "남자 회원 " << m << "명, 여자 회원 " << w << "명" << endl;
    altong.at("남") = 100; //at 함수가 반환한 참조를 통해 남자 회원 수를 100으로 수정
    m = altong.at("남"); // 수정된 남자 회원 수를 가져와 m에 저장
    w = altong.at("여"); // 현재 여자 회원 수를 가져와 w에 저장
    //수정 후의 남자와 여자 회원 수를 출력
    cout << "남자 회원 " << m << "명, 여자 회원 " << w << "명" << endl;

    return 0; //프로그램을 종료하고 운영체제에 0을 반환
}
```

실행결과



작성 소감

이번 과제를 작성하는 데 걸린 시간은 약 2시간 30분 정도였습니다.

6번 실습문제는 다각형 좌표를 관리하는 Polygon 프로그램 만들기였습니다.

먼저 4장에서 학습한 객체 배열의 동적 생성 개념을 적용해 보았습니다. 객체 생성 시점에 필요한 점의 개수를 파악하고, new 연산자로 xList와 yList 동적 배열을 생성함으로써 실행 중에 메모리를 효율적으로 할당하는 방식을 익힐 수 있었습니다.

사용이 끝난 후 소멸자에서 delete[]를 호출하여 할당받은 메모리를 잊지 않고 해제하는 것이 메모리 누수를 방지하는 핵심이라는 점을 다시 한번 확인했습니다.

또한 get() 함수를 구현하며 참조 매개변수의 유용함을 크게 느꼈습니다. 보통 함수는 return을 사용하여 하나의 값만 반환할 수 있지만, 참조 매개변수를 활용하니 함수 내부에서 수정한 x, y 좌표값이 호출한 곳의 변수에 그대로 반영되어 여러 개의 값을 동시에 전달하는 효과를 낼 수 있었습니다.

8번 실습문제는 피트니스 센터의 회원수를 관리하는 Fitness 프로그램 만들기였습니다.

문제에서는 arrive()함수를 한 줄에 여러 번 호출하는 방식을 사용했습니다. 이를 구현하기 위해 함수의 반환 타입을 클래스의 참조형으로 설정하고, 함수 마지막에 객체 자신의 참조인 return *this; 를 사용하여 arrive()함수를 구현할 수 있었습니다.

또한 at() 함수에서도 참조 리턴 int& 을 활용하여 멤버 변수의 값을 직접 수정하는 과정을 통해 참조의 활용성이 많다는 것을 알 수 있었습니다.

그리고 피트니스 프로그램을 구현하며 예외 상황 처리에 대해서도 고민했습니다. arrive() 함수와 count() 함수에서 잘못된 성별이 입력되면 에러 메시지를 출력하고, 특히 count() 함수에서는 인원수를 0으로 초기화하여 잘못된 성별이 입력된 곳에 안전한 값을 받도록 처리했습니다.

반면 at() 함수는 참조를 반환해야 하는 특수한 경우였습니다. 참조 리턴은 변수 자체를 반환해야 하는데, 잘못된 성별이 입력되었을 때 유효하지 않은 참조를 반환하면 엉뚱한 값이 출력되거나 프로그램이 불안정해질 수도 있겠다 생각했습니다. 따라서 at()함수만은 exit(1)을 사용하여 잘못된 성별이 입력되었을 때 프로그램을 종료하도록 구현했습니다.

이번 과제를 수행하며 단순히 문법을 암기하는 것을 넘어, 데이터를 객체 단위로 묶어 관리하는 방식이 얼마나 효율적인지 실감할 수 있었습니다. 헤더파일과 구현파일을 분리하여 작성하는 과정을 통해 코드의 구조를 훨씬 깔끔하게 구현할 수 있었습니다. 처음에는 복잡하게 느껴졌던 연쇄 호출이나 참조 리턴 같은 개념들이 실제 코드로 구현되어 동작하는 모습을 보며, 객체 지향 프로그래밍이 대규모 프로그램을 설계할 때 왜 유리한지 깊이 이해할 수 있었던 유익한 시간이었습니다.